İZMİR UNIVERSITY
OF ECONOMICS

Sport Management Simulation Game

Design and Implementation Report

Prepared by:

Şilan ATÇI	20230602008
Burak ÇALIŞKAN	20230602017
Beray KAYA	20230602045
Duru Eda CAN	20240602019

Instructor:
Doç. Dr. Kaya OĞUZ

Department of Computer Engineering

CE 216 Project

March 2026

Contents

1 Introduction

In the first milestone (M1), we designed the Sports Management Simulation System using a layered architecture and a sport-independent structure. At that stage, the system was mostly theoretical and based on UML diagrams.

In this milestone (M2), our goal is to turn that design into a working system. We implemented the main components of the system in Java and made sure that they worked together correctly. The system now supports basic operations such as creating a league, managing teams, and simulating matches through the GameFacade.

Another important goal of this milestone is to test whether our architecture is flexible and works as expected. For this reason, we implemented a football plugin as an example to show that different sports can be added without changing the core system.

We also focused on writing meaningful test cases to verify that the system behaves correctly. These tests help us check both normal scenarios and possible edge cases.

Overall, this milestone shows that our initial design is not only theoretical but also practical and implementable.

2 Changes

2.1 Domain Layered Architecture

2.1.1 Overview

In M1, the domain layer was designed at a conceptual level and mainly focused on defining core entities such as Season, League, Team, Player, Fixture, Round and Match, along with their relationships. These were described using UML diagrams, but they did not include detailed behavior or full implementation.

In M2, these classes were fully implemented in Java and the domain layer became more detailed. Each class now has clear responsibilities and includes the logic required for the simulation. Additionally, new domain concepts were introduced. The ManagerProfile class was added to represent the player's progress, including reputation and season progression, which are used to unlock features like hiring better coaches.

We also introduced several enum types to improve clarity and control, such as MatchStatus, InjuryStatus, TrainingIntensity, PlayStyle, and Gender. These help prevent invalid values and make the system easier to manage.

Validation checks were also added across the domain layer to prevent incorrect states, such as duplicate players or invalid matches. Another improvement was renaming the scheduling unit in Fixture from "Round" to "MatchWeek" to better reflect weekly matches. Extra features like match status tracking, duplicate prevention, and week completion checks were also added. The core idea remains the same: a group of matches played within one week.

Overall, the domain layer evolved from a simple conceptual design into a detailed and functional system that supports the core mechanics of the game.

2.1.2 Core Entities

The core entities such as Season, League, Team, Player, Fixture, and Match were originally defined in M1 and are not repeated here in detail. These entities still form the foundation of the system.

In M2, these entities were implemented as fully functional classes with added behaviors, validations, and interactions. Additional entities such as ManagerProfile were also introduced to support gameplay mechanics and progression.

2.1.3 Domain Relationships

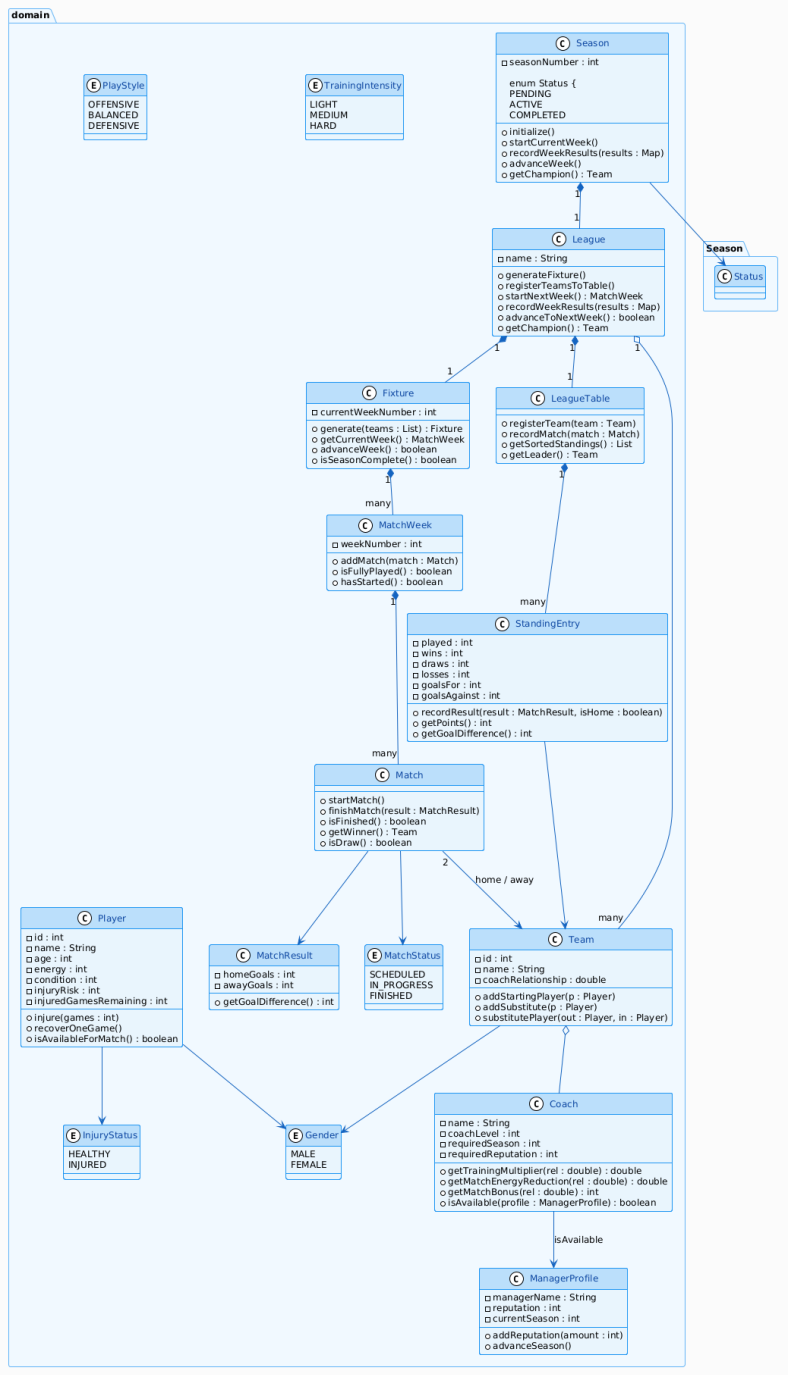

Figure 2.1: Domain Model

PlayStyle and TrainingIntensity are defined in the domain layer but mainly used in higher layers. Therefore, they are shown without direct associations in the diagram.

2.2 Sport Plugin Architecture

2.2.1 Overview

In M1, the sport plugin layer was designed using interfaces such as ISport, MatchSimulator, ScoringRule, TieBreakerRule, MatchFlow, and RosterRule. In M2, all of these interfaces were fully implemented through a football plugin.

The ITactic interface was also added to support sport-independent tactic definitions, implemented in the football plugin as FootballTactic. This structure allows adding new sports without changing the core system.

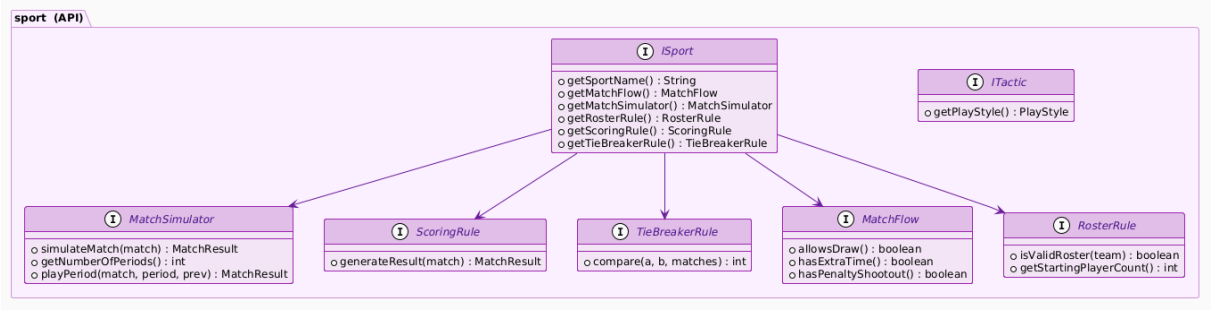

Figure 2.2: Sport API Layer

2.2.2 Example Sport (Football)

A football implementation was developed to demonstrate the plugin structure. FootballSport acts as the entry point and connects all Sport API components. Match simulation, scoring, ranking, and roster validation are handled by FootballMatchSimulator, FootballScoringRule, FootballTieBreakerRule, FootballRosterRule, and FootballMatchFlow respectively.

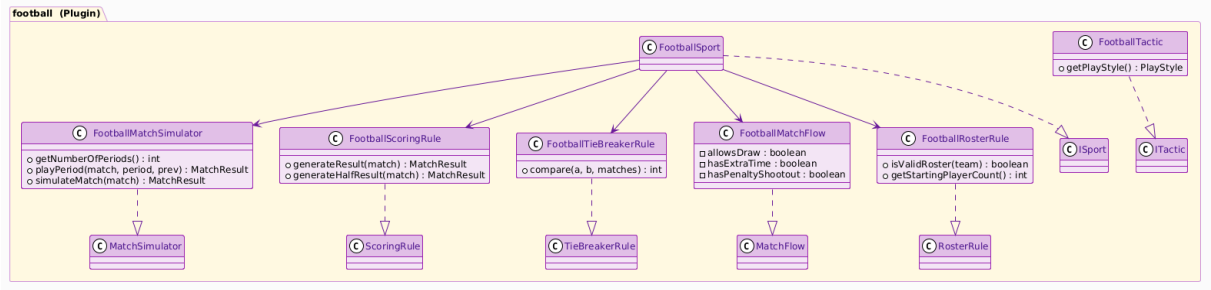

Figure 2.3: Football

2.3 Application Layer

2.3.1 Overview

In M1, the application layer was only defined conceptually and focused on structuring the interaction between the UI and the domain layer. In M2, this layer was implemented and expanded with multiple classes that manage the core logic of the system. The application layer acts as a bridge between the UI and the domain, coordinating operations without exposing internal details. The design is modular and follows separation of concerns, where each class has a clear responsibility.

2.3.2 GameFacade

The GameFacade provides a single entry point for the UI to interact with the system. Instead of directly accessing domain or manager classes, all operations are performed through this interface. The GameFacade interface defines available operations, while DefaultGameFacade coordinates these operations by delegating tasks to the appropriate manager classes. This reduces coupling between the UI and the system and makes the architecture easier to maintain and extend.

2.3.3 Managers

Manager classes are responsible for handling specific parts of the system logic. Each manager focuses on a single responsibility. For example, LeagueManager manages league creation and structure, TeamManager handles team-related operations such as creating teams, adding players, and transfers, and MatchManager is responsible for match-related operations. PlayerManager manages player storage and retrieval, while TrainingManager applies training effects based on intensity and coach influence. This structure improves maintainability and follows SOLID design principles.

In this context, SeasonCycleManager was introduced to manage the season lifecycle and transitions between seasons. Although this responsibility was not clearly assigned in M1, during implementation it became clear that coordinating season state, league creation, and week progression required a dedicated component.

2.3.4 Generators

During implementation, we realized that putting object creation logic inside manager classes would violate the Single Responsibility Principle. To keep each class focused on a single concern, we introduced two dedicated generator classes. PlayerGenerator is responsible for creating Player objects with randomized or constrained attributes such as gender, age, energy, and condition. TeamGenerator builds fully initialized AI teams by combining player generation, coach assignment, tactic selection, and chemistry initialization in a single controlled operation. This separation ensures that manager classes remain focused on coordination logic rather than object construction.

2.3.5 Utility

As the number of generated names grew, we needed a centralized and consistent source for player and team names. NamePool was introduced as a static utility class that provides predefined name lists used by the generator classes. This avoids scattered string literals across the codebase and makes name management easier to maintain.

2.3.6 Application Layer Relationships

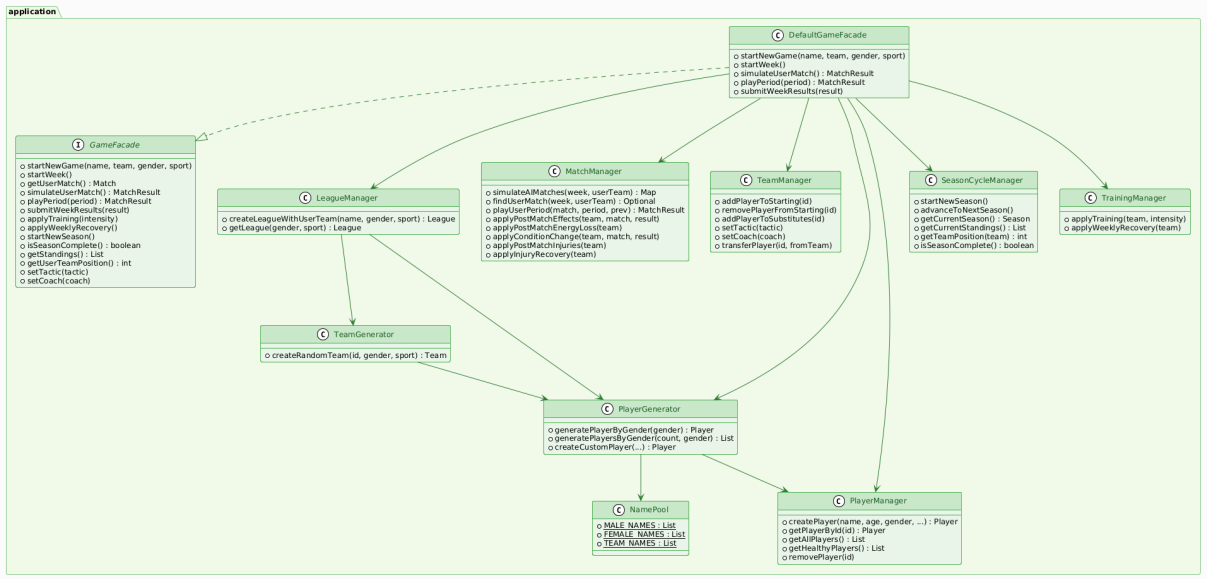

Figure 2.4: Application Layer

2.4 Simulation Flow

The system follows a structured flow to simulate a sports season:

First, a new game is initialized through the GameFacade. This creates the user team and generates AI teams using TeamGenerator. Then, a league is created and all teams are registered.

The season starts by generating a fixture and organizing matches into weekly rounds. Each week, the system progresses by applying training, simulating matches, and updating the league table.

Match simulation is handled by the sport plugin layer. The MatchManager calls the MatchSimulator, which uses sport-specific rules such as scoring, match flow, and tie-breaking logic.

During the simulation, managers coordinate different parts of the system. For example, TeamManager manages team structure, PlayerManager handles players, and TrainingManager applies training effects.

This flow continues until all weeks are completed. At the end of the season, the system determines the champion based on the league standings.

Overall, the system demonstrates a modular simulation where each layer has a clear responsibility.

2.4.1 Layer Relationships

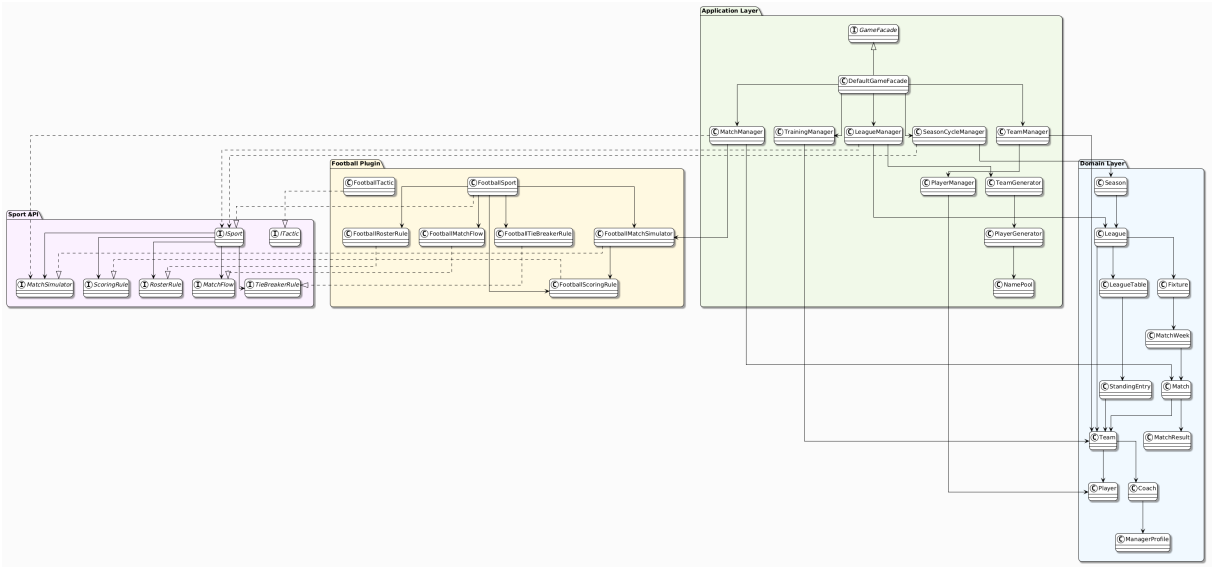

Figure 2.5: General UML

2.4.2 Match Simulation Sequence

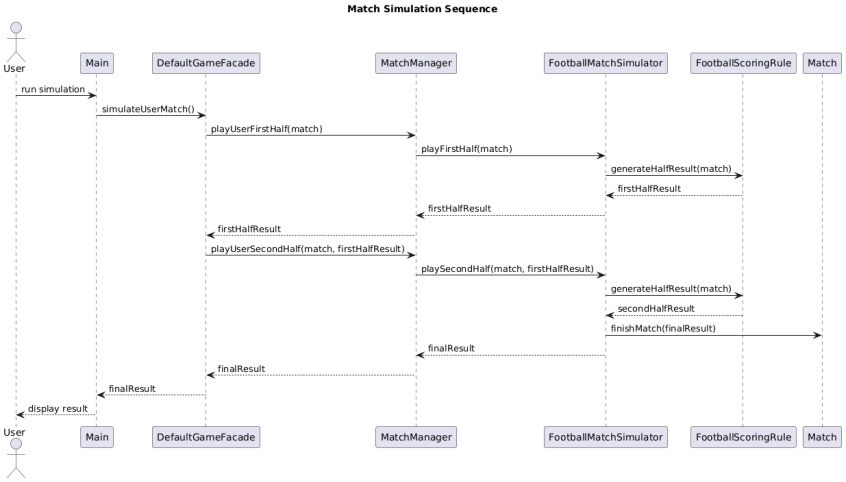

Figure 2.6: Match Simulation Sequence Diagram

Compared to M1, the match simulation sequence was significantly refined. The original design showed a simplified flow from GameFacade directly to Match and ISport. In M2, the actual implementation introduced MatchManager as an intermediate coordinator and split the simulation into two periods, each handled by FootballMatchSimulator and FootballScoringRule separately.

3 Tests

Unit tests were written using JUnit 5 with no mocking framework. In total, 23 test cases were implemented across 6 test classes, grouped by layer.

3.1 Domain Layer Tests

The domain layer tests focus on verifying the core game logic, validation rules, and object behaviors. A total of 15 test cases were implemented in this layer.

3.1.1 CoachTest

- **constructorSetsFieldsCorrectly:** Verifies that all coach attributes are initialized correctly.
- **trainingMultiplierScalesWithLevelAndRelationship:** Confirms that the training multiplier increases based on coach level and player relationship (from 1.0 up to 1.8).
- **matchEnergyReductionCappedAtMax:** Ensures that energy reduction provided by the coach does not exceed the maximum limit.
- **isAvailableChecksSeasonAndReputation:** Verifies that a coach becomes available only when the manager meets the required season and reputation conditions.

Analysis: The Coach class affects both training efficiency and energy management. Tests confirm that scaling works correctly and that availability conditions are enforced.

3.1.2 PlayerTest

- **constructorSetsFieldsCorrectly:** Verifies correct initialization of player attributes.
- **constructorRejectsInvalidParameters:** Ensures invalid inputs (e.g., null name, negative values) are rejected.
- **setEnergyEnforcesBounds:** Checks that energy values remain within valid limits (0–100).
- **injureSetsStatusAndCountdown:** Verifies that injury correctly updates player status and recovery duration.
- **recoverOneGameDecrementsAndHeals:** Ensures that recovery progresses correctly and eventually heals the player.
- **isAvailableForMatchWhenHealthyWithEnergy:** Checks that a player is available when healthy and has enough energy.

- **isAvailableForMatchReturnsFalseWhenInjured:** Ensures that injured players cannot participate in matches.

Analysis: The Player class enforces strict validation rules. Tests confirm data integrity and verify that the injury and recovery system works correctly. Player availability depends on both health and energy.

3.1.3 TeamTest

- **addStartingPlayerAppearsInList:** Ensures that adding a player correctly updates the starting lineup.
- **addStartingPlayerRejectsGenderMismatch:** Verifies that players with mismatched gender cannot be added.
- **substitutePlayerSwapsCorrectly:** Ensures that substitution correctly swaps players.
- **addDuplicatePlayerThrowsException:** Verifies that duplicate players cannot be added.

Analysis: The Team class ensures consistency in team structure. Tests confirm that rules such as gender constraints, substitutions, and duplicate prevention are correctly enforced.

3.2 Football Plugin Tests

The football plugin tests focus on verifying scoring logic and ensuring realistic match outcomes.

3.2.1 FootballScoringRuleTest

- **generateHalfResultReturnsNonNegativeGoals:** Verifies that generated goal values are never negative.
- **injuredPlayersExcludedFromAverages:** Ensures that injured players do not affect team performance calculations.

Analysis: Goal values never drop below zero, confirming that the algorithm works within a safe range. Only healthy players are considered in calculations, which prevents unrealistic results. Teams with fewer healthy or low-energy players produce weaker outcomes, reflecting realistic match behavior.

3.3 Application Layer Tests

The application layer tests focus on verifying how match results and training processes affect players. A total of 6 test cases were implemented.

3.3.1 MatchManagerTest

- **conditionChangeOnWinIsNetZero:** Verifies that player condition does not change after a win.
- **conditionChangeOnLossReducesCondition:** Checks that player condition decreases after a loss.
- **postMatchEnergyLossReducedByCoach:** Verifies that a good coach reduces post-match energy loss.

3.3.2 TrainingManagerTest

- **hardTrainingIncreasesConditionDecreasesEnergy:** Verifies that hard training increases condition, decreases energy, and increases injury risk.
- **trainingSkipsInjuredPlayers:** Ensures that injured players are not affected by training.
- **weeklyRecoveryGivesSubsMoreEnergy:** Verifies that substitutes recover more energy than starting players.

Analysis: These tests confirm that match outcomes and training correctly affect player attributes. They also ensure that rules such as coach effects, condition changes, and injury handling are applied consistently.

4 Conclusion and Future Work

4.1 General Work

In the first milestone, we established the theoretical foundation of the Sports Manager system through UML diagrams and a layered architecture design. In this milestone, that design was translated into a fully working Java implementation.

All interfaces defined in the Sport API layer were implemented, and the Football plugin was developed as a concrete proof that the plugin architecture functions as intended. The domain layer was expanded with validation logic and lifecycle management, while the application layer grew with additional classes introduced to maintain a clean separation of responsibilities. A total of 23 test cases were written to verify correctness across all layers.

Overall, M2 demonstrates that the architecture designed in M1 works as expected and can be implemented successfully.

4.2 Future Work

The primary goal for M3 is to introduce a graphical user interface built with JavaFX. Since all operations go through the GameFacade interface, the UI can be connected without modifying any existing layer. Additionally, a second sport will be implemented to demonstrate that the system is not dependent on a single sport and that the plugin architecture works as intended.

4.3 Technical Requirements

4.3.1 Development Environment

The project is developed using Apache Maven as the build and dependency management tool. The development environment is configured with the following versions:

- **Java:** OpenJDK 26
- **Maven:** 3.9.13
- **JUnit:** 5.10.2

4.3.2 Build and Execution

The project lifecycle is managed through Maven. The following commands are used for compilation, execution, and testing:

```
mvn compile
mvn exec:java
mvn test
```

The main entry point of the application is defined in the `pom.xml` file using the Maven Exec Plugin as `application.Main`.

4.3.3 Execution Results

The `mvn compile` command builds the project successfully without errors.

BUILD SUCCESS

```
[INFO] Building sports-manager 1.0-SNAPSHOT
[INFO] Nothing to compile - all classes are up to date
[INFO] BUILD SUCCESS
[INFO] Total time: 0.235 s
```

The `mvn exec:java` command runs the application and starts the season simulation.

SIMULATION OUTPUT

```
[INFO] --- exec:3.1.0:java (default-cli) @ sports-manager ---
=======================================================
         SPORTS MANAGER - SEASON SIMULATION
=======================================================
Manager      : Ahmet
Team         : Eagles FC
Coach        : Default Coach (Level 1)
Tactic       : BALANCED
Season       : 1 | Total Weeks: 34

--- Eagles FC Roster ---
Name                Role   Energy  Condition  Status
-------------------------------------------------------------------
Christopher Murphy  START  84      67         HEALTHY
Ozan Tekin          START  87      84         HEALTHY
Diego Fernandez     START  97      95         HEALTHY
...
```

The `mvn test` command executes all unit tests using JUnit. All tests pass successfully.

TEST RESULTS

```
[INFO] Building sports-manager 1.0-SNAPSHOT
[INFO] Tests run: 23, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 0.235 s
```

4.3.4 Summary

The successful execution of build, run, and test commands demonstrates that the system is fully functional and properly integrated using Maven.